NCUSCC 秋季招新考核 · 技术报告

> 仓库地址：**https://github.com/wwWWW56600/nucscc-cpp**

第 0 章 仓库目录结构说明

最终交付的 GitHub 仓库结构如下：

```
nucscc-cpp（https://github.com/wwWWW56600/nucscc-cpp）
├── README.md                # 项目介绍 + 环境 + 构建运行方式 + 目录说明 + 命令行清单（含真实
├── .gitignore               # 忽略构建产物与工作区散件（含锚定根目录的规则）
└── src/                     # 全部源码，按关分目录；每关子目录自带 README.md（构建/运行/样例）
    ├── L1_cpp/              # 读数统计
    ├── L2_cpp/              # 设备巡检记录
    ├── L3_cpp/              # 固定容量读数窗口（环形缓冲）
    ├── L4_cpp/              # 多文件统计工具（CMake 多文件构建）
    └── L5_cpp/              # 读数变化观察（相邻差分计数）
```

第 1 章 环境与本机工具链

- **考核平台**：网页 IDE（左侧文件树 / 中间终端 / 右侧查看器），底层为 Linux 容器，预装 `g++`、`cmake`、`exam` 等工具。
- **编译链**：C++ 关使用 `g++ -std=c++17`；多文件关使用 `cmake -S . -B build && cmake --build build` 生成 `build/main`。
- **本机上传工具**：Git for Windows。
- **关键命令清单**：
- `cat > 文件`：把后续粘贴内容写入文件（覆盖），`Ctrl+D` 结束。
- `g++ -std=c++17 -o main main.cpp`：按 C++17 编译生成可执行 `main`。
- `./main < 样例.in`：把样例文件重定向为程序标准输入。
- `diff <(命令) 期望.out && echo OK`：比对实际输出与期望输出，无差异即静默通过。
- `rm -rf build`：删除旧构建目录，强制干净重建。
- `cmake -S . -B build` / `cmake --build build`：配置并构建工程。

第 2 章 Git 与 GitHub 探索使用过程

2.1 真实执行流程（以下均为实际发生的操作）

① 本地建仓与首次提交（Git Bash，工作区根目录）：

```
git init
git add README.md src .gitignore 上传说明.md upload.bat
git commit -m "NCUSCC 招新考核：进阶 C/C++ L1-L5 通关源码 + 上传说明"
```

② **GitHub 网页端建仓**：新建空仓库 `nucscc-cpp`（Public，不勾选 README/.gitignore/License 初始化，避免与本地首提交冲突）。

③ **关联远端并推送**：

```
git branch -M main
git remote add origin https://github.com/wwWWW56600/nucscc-cpp.git
git push -u origin main
```

推送成功输出：`* [new branch] main -> main`、`branch 'main' set up to track 'origin/main'`。建立跟踪关系后，本地后续 `git push` / `git pull` 不必再写远端名与分支名。

2.2 过程中的真实问题与排查

+ **问题：push 连不上 GitHub。** 首次推送报 `Failed to connect to github.com:443 after 21101 ms: Could not connect to server`——浏览器能打开 GitHub（走了 Clash Verge 代理），但 git 命令行不读系统代理。解决：在上传脚本内设置进程级环境变量 `HTTP_PROXY` / `HTTPS_PROXY=http://127.0.0.1:7897`（Clash Verge 混合端口），仅脚本运行期间生效、不改全局 git 配置；重跑后推送成功。

2.3 最终提交记录

commit	说明	内容
`ff6c8fc`	NCUSCC 招新考核：进阶 C/C++ L1-L5 通关源码 + 上传说明	首次提交：src/ 全关源码 + README + .gitignore + 上传脚本，共 19 个文件
`8c9fe89`	NCUSCC cpp L1-L5 source + upload guide	上传脚本补充代理配置后的增量提交
`823b023`	remove non-code files, keep code only	用 `git rm --cached` 从仓库移除辅助文件（zip 包/上传脚本/说明），补全 `.gitignore`，仓库仅留代码三件套

第 3 章 Shell 基础语法概念

- **重定向 `>` 与 `>>`**：`>` 将命令输出写入文件（覆盖），`>>` 追加。如 `cat > main.cpp` 把终端输入写进文件。

- **管道 `|`**：把前一命令的标准输出作为后一命令的标准输入，用于串联处理（如 `grep ... | sort`）。
- **变量与退出码 `$?`**：`$?` 取上一条命令的退出码，`0` 表示成功，非 `0` 表示失败；用于判断程序是否正常结束。
- **通配符与引号**：`*` 匹配任意字符；双引号内变量会被展开，单引号保持字面量，文件名含空格时必须引号包裹。
- **进程替换 `<()`**：把命令输出伪装成临时文件，便于 `diff <(程序输出) 期望文件` 做逐字符比对。

第 4 章 逐题记录

4.1 基础关

通用环境与工具清单（基础关三题共用） - 考核平台：网页 IDE（左侧文件树 / 中间终端 / 右侧查看器），底层 Linux 容器，预装 `exam` 处理程序与 `ls` / `cat` / `cp` / `find` / `grep` / `wc` / `head` / `printf` / `sed` 等标准工具。 - 核心程序 `exam`：固定读取 `/workspace/input`，以 `-e 密钥` 接收密钥，将结果写入 `/workspace/output`（写文件而非 stdout）；`exam -h` 可查看自身接口说明。内部对输入逐字节做 `value ^ key[i % len(key)]` 循环异或，约束 `len(data) % len(key) == 0`，`os.replace` 原子写，三态退出码（0 成功 / 1 输入异常 / 2 密钥长度不匹配）。

L1 找到入口

1. 题目详细描述 离线 Linux 工作区中，为一份待处理材料生成结果文件。材料位于 `/workspace/`，包括输入文件和密钥文本；处理程序为 `/workspace/exam`，调用接口以程序自身帮助信息为准。提交 `/workspace/flag`，须为大小不超过 1 MiB 的普通文件，内容与"输入经对应密钥和 exam 处理得到的结果"逐字节相同；结果按二进制字节比较。

2. 环境及工具清单 同通用环境。本关用到 `ls`、`cat`、`exam -h`、`exam -e`、`cp`、`ls -l`。

3. 实践过程和设计细节 - 黑盒建模：输入 = `/workspace/input`（固定路径）+ `-e` 后密钥；处理 = 循环异或；输出 = `/workspace/output`（落到文件）；提交 = 复制 output 为 flag。 - 流程：① `ls /workspace` 定位输入、密钥、`exam`；② `/workspace/exam -h` 读帮助，确认接口（`exam [-h] -e 密钥`，固定读 input、写 output）；③ `cat 密钥文件` 取密钥，原样用于 `-e`（无空格可省引号，含空格必须引号包裹）；④ `/workspace/exam -e "密钥"` 运行，自动生成 `/workspace/output`；⑤ `cp /workspace/output /workspace/flag` 复制为提交文件名（`cp` 逐字节一致）；⑥ `ls -l /workspace/flag` 校验为普通文件且 `< 1 MiB`。

L2 两段信号

1. 题目详细描述 一批材料中夹杂无关文件，需找出两段有效信号（文件名 `signal-A`、`signal-B` 标识，目录多层嵌套）并生成完整结果；对应密钥来自清单中唯一的 `ACCESS_KEY=` 记录（清单含干

扰）。处理程序 `/workspace/exam`。最终 `flag` 必须严格等于 A 段结果与 B 段结果的有序拼接（A 在前、B 在后），不能缺段、重复或增加额外字节，大小 ≤ 1 MiB。

2. 环境及工具清单 同通用环境。本关新增 `find`（递归查找）、`grep -r`（全局搜密钥）。

3. 实践过程和设计细节 - 黑盒建模：与 L1 同一 `exam`；本关有两份输入 + 含干扰的密钥清单；`exam` 固定输出 `/workspace/output` 会被后一次运行**覆盖**，须分别处理独立保存。 - 流程：① `find /workspace -name "*signal-A*"` / `"*signal-B*"` 递归定位（找不到用 `find /workspace -type f` 列全部肉眼找）；② `grep -r "ACCESS_KEY=" /workspace` 搜密钥行，**整行未必都是密钥**，只取 `=` 之后字符串；③ 分别处理独立保存（`/workspace/exam -e "密钥" signal-A > resA`，B 同理；或 `exam -e 密钥` 后 `mv output resA`）；④ 有序拼接 `cat resA resB > flag`（A 前 B 后，二进制无损）；⑤ `ls -l flag` 校验。 - 关键：`>` 覆盖、`>>` 追加必须分清；拼接顺序 A→B 不可颠倒。

L3 读日志修复输入

1. 题目详细描述 一份材料未被 `exam` 正确处理。需依据程序接口说明和诊断信息恢复处理并提交有效结果。原始输入和密钥位于 `/workspace/`，处理程序 `/workspace/exam`，诊断日志位于 `/var/log/ncuscc/exam.log`。原始文本为 ASCII 并带一个结尾换行。允许的修改**仅为**在"最后一行文字"与"结尾换行"之间插入 ASCII 空格，其他字节必须保留，且新增空格数最少。最终提交普通文件 `/workspace/flag`（≤1 MiB），内容与按上述限制修复输入后生成的结果逐字节相同；无需提交修改后的输入文件。

2. 环境及工具清单 同通用环境。本关新增 `wc -c`（统计字节）、`head -c -1`（删末尾字节）、`printf`（精确插入字节）。

3. 实践过程和设计细节 - 黑盒建模：输入因长度不满足 `exam` 约束（`len(data) % len(key) == 0`）运行失败；日志给出字节长度不匹配原因。修复 = 在「文字 ↔ 换行」间插最少空格使长度满足约束，再跑 exam 出 flag。 - 流程：① `cat /var/log/ncuscc/exam.log` 读诊断（退出码只说成败、日志说原因；日志由 exam 运行时才写，不跑则无）；② `ls` / `cat` / `wc -c` 看内容与**字节数**（结尾 `\n` 占 1 字节）；③ 由日志算需补空格 = 目标字节 − 当前字节（每空格 +1 字节）；④ 构造修复文件：`head -c -1 输入 > fixed_input`（删原换行腾位）→ `printf ' ' >> fixed_input`（插 N 空格）→ `printf '\n' >> fixed_input`（补回换行）→ `wc -c` 核对；⑤ `cp fixed_input /workspace/input` 后 `exam -e "密钥"` → `cp output flag`；⑥ 校验 `< 1 MiB`。 - 实例：原始 `REPAIR THIS SIGNAL\n` 共 19 字节，密钥 `NOVA`（长 4），`19 % 4 = 3` → 插 1 空格使总长 20 = 4×5，再 `exam -e NOVA` 得结果。

4.2 进阶 C/C++ 关（L1–L5）

通用环境与工具清单（五题共用） - 考核平台：网页 IDE（左侧文件树 / 中间终端 / 右侧查看器），底层 Linux 容器，预装 `g++`、`cmake`。 - 编译器：`g++`（按 C++17，`-std=c++17`）；本机验证用 `C:/msys64/mingw64/bin/g++.exe`（同 GCC 工具链）。 - 终端命令：`cat > 文件`（写文件，`Ctrl+D` 结束）、`g++ ... -o main`、`./main < samples/x.in`（输入重定向）、`diff <(./main < in) out`（进程替换比对）、`echo $?`（退出码）。

L1 cpp • 读数统计

1. 题目详细描述 从标准输入读第一行整数 `n`，第二行 `n` 个整数（空格分隔，可负）。程序须向标准输出打印**恰好四行**：`count=<n>`、`sum=<总和>`、`min=<最小值>`、`max=<最大值>`，每行以 `\n` 结尾；正常退出码 0、读入失败返回 1。评测用三组公开样例（混合正负数、全负、含 0 等）加隐藏用例逐字节比对。

2. 环境及工具清单 同通用环境。

3. 实践过程和设计细节 - 黑盒建模：输入契约 = `n` + `n` 个整数（走 stdin，无命令行参数）；输出契约 = 固定四行前缀 + 数值 + 末尾换行。 - 设计要点（见 `src/L1_cpp/main.cpp`）：① `min` / `max` 用**读到的第一个数**初始化，而非用 `0` ——否则全负数样例会被误判最小值为 0；② `sum` 用 `long long` 防大数溢出（防御性写法，本题 `int` 也够但保留 64 位更稳）；③ 循环读 `v` 时首数特判初始化，其余 `min=min(min,v)` / `max=max(max,v)` 并 `sum+=v`。 - 验证：本地 `g++ -std=c++17 -Wall -Wextra -o main main.cpp` 零警告；`diff <(./main < samples/01-mixed.in) samples/01-mixed.out && echo OK` 三组样例全绿后提交。

L2 cpp • 设备巡检记录

1. 题目详细描述 读 `n`，随后 `n` 行，每行 `type status hours`；`type` ∈ `{printer,scanner,router,server}`，`status` ∈ `{ok,repair}`，`hours` 为整数。按**固定顺序**输出四行 `<type> <count> <hours>`（即便某类数量为 0 也必须输出 `printer 0 0` 这类），末行 `busiest <type>` = 累计工时最大的类型；若多类并列最大工时，取**固定顺序中靠前的下标**。退出码 0。

2. 环境及工具清单 同通用环境。本关用 `for f in samples/*.in; do diff ...; done` 批量对样例，及 `sed -i '/\x1b/d'` 清 ESC 行。

3. 实践过程和设计细节 - 黑盒建模：输入为结构化记录流；处理 = 按类型聚合（计数 + 工时累加，`status` 读出但不影响统计）；输出 = 固定四行报表 + 最繁忙类型。 - 设计（见 `src/L2_cpp/main.cpp`）：① 长度 4 的数组 `byType[]`，下标即类型固定顺序（0=printer…），遍历即按序输出，避免手工 if 分支；② `typeIndex()` 映射字符串到下标，未知类型返回 `-1` 并 `return 1` 防越界；③ 工时**无条件累加**（`status` 仅用于正确解析每行三字段）；④ `busiest` 用 `best` 初值 0 + 仅 `>` 更新（不用 `>=`），保证并列保留靠前下标。 - 验证：样例 `01-mixed`（常规）、`02-tie`（并列取靠前）、`03-empty-types`（空类别也要 `0 0`）三组 diff 全 OK。

L3 cpp • 固定容量读数窗口（环形缓冲）

1. 题目详细描述 读 `n`（1≤n≤100000）与第二行 `n` 个整数（[-1000,1000]）。程序维护**容量 8** 的窗口，只保留最近 8 个读数；输出恰好四行 `kept=`（实际保留个数，封顶 8）、`oldest=`（最旧值）、`last=`（最新值）、`sum=`（窗口内求和）。当 n≤8 时保留全部（oldest=window[0]、last=window[n-1]）；当 n>8 时窗口满且环形排列（oldest=window[n%8]、last=window[(n-1)%8]、sum 加全部 8 项）。

2. 环境及工具清单 同通用环境。

3. 实践过程和设计细节 - 黑盒建模：输入 = 长序列；处理 = 定长 8 格窗口 + 回绕写指针；输出 = 受容量约束的四项汇总。 - 设计（见 `src/L3_cpp/main.cpp`）：① 定长 `window[8]`，写入第 `i` 个读数落到 `i % 8`，实现"保留最近 8 个、自动覆盖最旧"；② `kept` 封顶 8（`if(kept<8)++kept`）；③ 输出段**也按环形取**：`oldest=window[n%8]`、`last=window[(n-1)%8]`、`sum` 从起点绕回累加 `kept` 项（起步代码用 `window[kept-1]` 在 kept>8 时会越界读，需重写为环形）。

L4 cpp • 多文件统计工具（CMake 构建）

1. 题目详细描述 把"统计功能"拆成入口（`main`）、统计（`stats`）、格式化（`format`）三个模块；初始工程能读但不能正确构建/输出。需修复两类问题：① 构建入口未包含全部实现文件；② 格式化函数声明与定义签名不一致。要求干净构建生成 `build/main`，按契约输出四行 `count=/sum=/min=/max=`，`sum` 必须 64 位。提交时根目录含 `CMakeLists.txt` + 全部源文件，**不提交 `build/` 预编译产物。**

2. 环境及工具清单 在通用环境基础上新增 **CMake**：`cmake -S . -B build`（配置，读 CMakeLists.txt 生成构建系统到 build/）与 `cmake --build build`（按配置编译链接出 build/main）。 `rm -rf build` 用于干净重建。

3. 实践过程和设计细节 - 黑盒建模（多文件拆分）：stdin → `main` 读入 → `stats::compute` 算 `Statistics{count,sum,min,max}`（全 long long）→ `format::print_report(count,total,smallest,largest)` 按固定顺序输出四行 → stdout。 - 设计（见 `src/L4_cpp/`）：① `stats.h/cpp` 定义结构体与 `compute`；② `format.h/cpp` 声明/定义 `print_report`，参数顺序固定为 `(count,total,smallest,largest)`，均 long long；③ `CMakeLists.txt` 用 `add_executable(main main.cpp stats.cpp format.cpp)` **列全三文件**；④ 头文件加 `#ifndef` 守卫防重复包含。 - 验证：`rm -rf build && cmake -S . -B build && cmake --build build` 构建零报错；三组样例（01-wide 含 ±1e9、02-overflow sum=5e9、03-zero）diff 全 OK。

L5 cpp • 读数变化观察

1. 题目详细描述 读 `n` 与第二行 `n` 个整数（[-1000,1000]）。从**第二项起**与紧邻前一项比较：`>` 则 `increases+1`，`<` 则 `drops+1`，`==` 两者都不变；第一项无前项，**只记账不比较**；`sum` 累加所有值。输出恰好四行 `count=/sum=/increases=/drops=`，退出码 0。

2. 环境及工具清单 在通用环境基础上新增 **GDB 调试链**：`g++ -std=c++17 -g -O0 -o main main.cpp`（加调试信息 + 关优化）后 `gdb ./main` 打断点观察；以及 `sed -i '/\x1b/d'` 清 ESC。

3. 实践过程和设计细节 - 黑盒建模：输入 = 序列；处理 = 相邻差分统计（当前项 vs 前一项）；输出 = 四行汇总。 - 设计（见 `src/L5_cpp/main.cpp`）：① `long long sum` 防溢出；`prev` 记前一项；② 循环 `for i in [0,n)`：读 `v`；`i==0` 时仅 `prev=v; sum=v; continue`（**跳过比较**）；其后 `sum+=v`，按 `v` 与 `prev` 大小更新计数，再 `prev=v`；等价写法：循环从 `i=1` 起、`prev=data[0]`。 - 验证：样

例 1（混合）`count=6/sum=24/increases=2/drops=2`；样例 2（单项）increases=0；样例 3（全相等）increases=0/drops=0。

第 5 章 小结

- 知晓了报告格式及优化方法，guthub仓库框架。
- 对于git使用中常见问题有一定了解，熟悉git,linux的操作。
- 学会了如何使用建立guthub仓库及上传文件，README的格式。
